

计算机系统 3 复习

add, sub 溢出条件

操作	操作数A	操作数B	表示结果有溢出的条件
$A+B$	≥ 0	≥ 0	< 0
$A+B$	< 0	< 0	≥ 0
$A-B$	≥ 0	< 0	< 0
$A-B$	< 0	≥ 0	≥ 0

图 3-2 加减法的溢出条件

I 型、R 型和 J 型指令的格式（字段的划分和字段名）

MIPS指令（add/sub/addi/lw/sw）格式汇总（含指令示例）

Name	Format	Example						Comments
add	R	0	18	19	17	0	32	add \$s1,\$s2,\$s3
sub	R	0	18	19	17	0	34	sub \$s1,\$s2,\$s3
addi	I	8	18	17	100			addi \$s1,\$s2,100
lw	I	35	18	17	100			lw \$s1,100(\$s2)
sw	I	43	18	17	100			sw \$s1,100(\$s2)
Field size		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions are 32 bits long
R-format	R	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	I	op	rs	rt	address			Data transfer format

■ 从指令编码产生控制信号

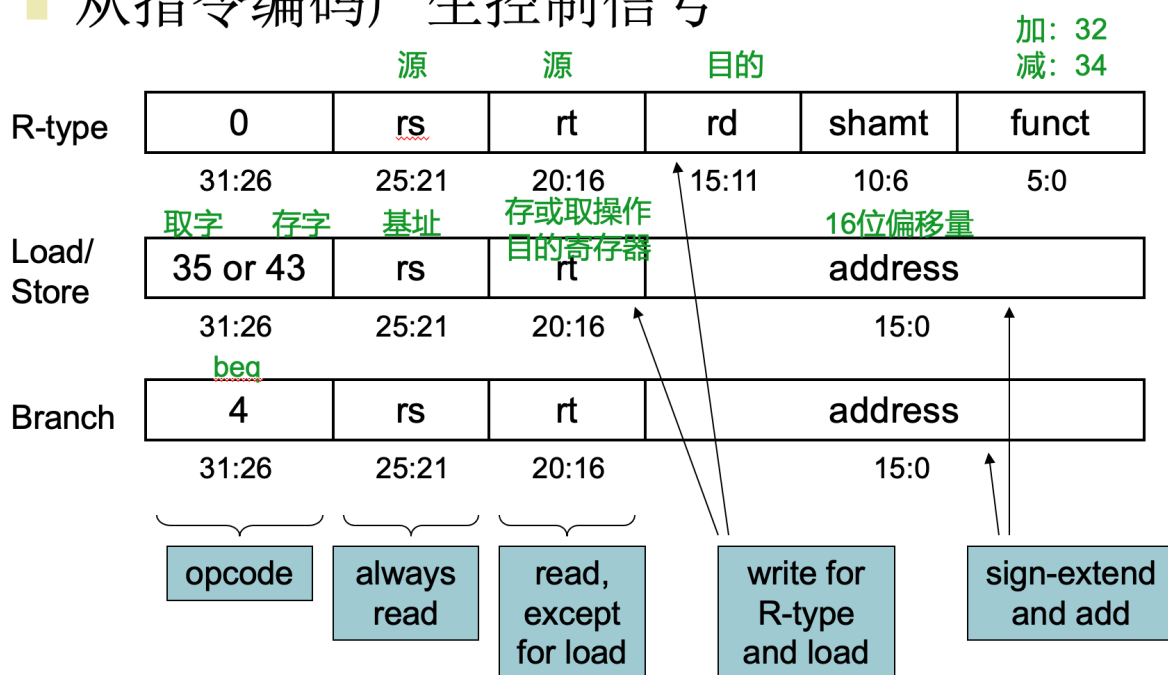


图 4-14

寄存器

MIPS寄存器约定

Name	Register number	Usage	Preserved on call?
\$zero	0	The constant value 0	n.a.
\$v0-\$v1	2-3	Values for results and expression evaluation	no
\$a0-\$a3	4-7	Arguments	no
\$t0-\$t7	8-15	Temporaries	no
\$s0-\$s7	16-23	Saved	yes
\$t8-\$t9	24-25	More temporaries	no
\$gp	28	Global pointer	yes
\$sp	29	Stack pointer	yes
\$fp	30	Frame pointer	yes
\$ra	31	Return address	yes

ALU控制信号

opcode	ALUOp	Operation	funct	ALU function	ALU control
lw	00	load word	XXXXXX	add	0010
sw	00	store word	XXXXXX	add	0010
beq	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
		subtract	100010	subtract	0110
		AND	100100	AND	0000
		OR	100101	OR	0001
		set-on-less-than	101010	set-on-less-than	0111

■ ALU控制的真值表

ALUOp		Funct field						Operation
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

图 4-13

无关项

浮点数结构

IEEE 浮点数标准格式

单精度single: 8 bits

单精度single: 23 bits

双精度double: 11 bits

双精度double: 52 bits

S	阶码	尾数
---	----	----

$$x = (-1)^S \times (1 + \text{尾数}) \times 2^{(\text{阶码} - \text{偏移})}$$

- S: 符号位 (0 $\Rightarrow$ 非负数, 1 $\Rightarrow$ 负数)
- 有效位的规格化: $1.0 \leq |\text{有效位}| < 2.0$
 - 数前总有一个前导的1, 此外作为隐含位可以不表示。
 - 有效位是"1.尾数"
- 阶码: 移码表示: 真实指数 + 偏移
 - 确保阶码非负
 - 单精度: 偏移 = 127; 双精度: 偏移 = 1203

控制信号逻辑实现

控制信号的逻辑实现

- 输入为opcode
- 控制信号真值表

图 4-22

Input or output	Signal name	R-format	lw	sw	beq
Inputs	Op5	0	1	1	0
	Op4	0	0	0	0
	Op3	0	0	1	0
	Op2	0	0	0	1
	Op1	0	1	1	0
	Op0	0	1	1	0
Outputs	RegDst	1	0	X	X
	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

其中
ALUOp0/ALUOp1
的编码见图4-13

Chapter 4 — The Processor — 40

CPI

CPU时钟周期数 = 程序的指令数 × 每条指令的平均时钟周期数

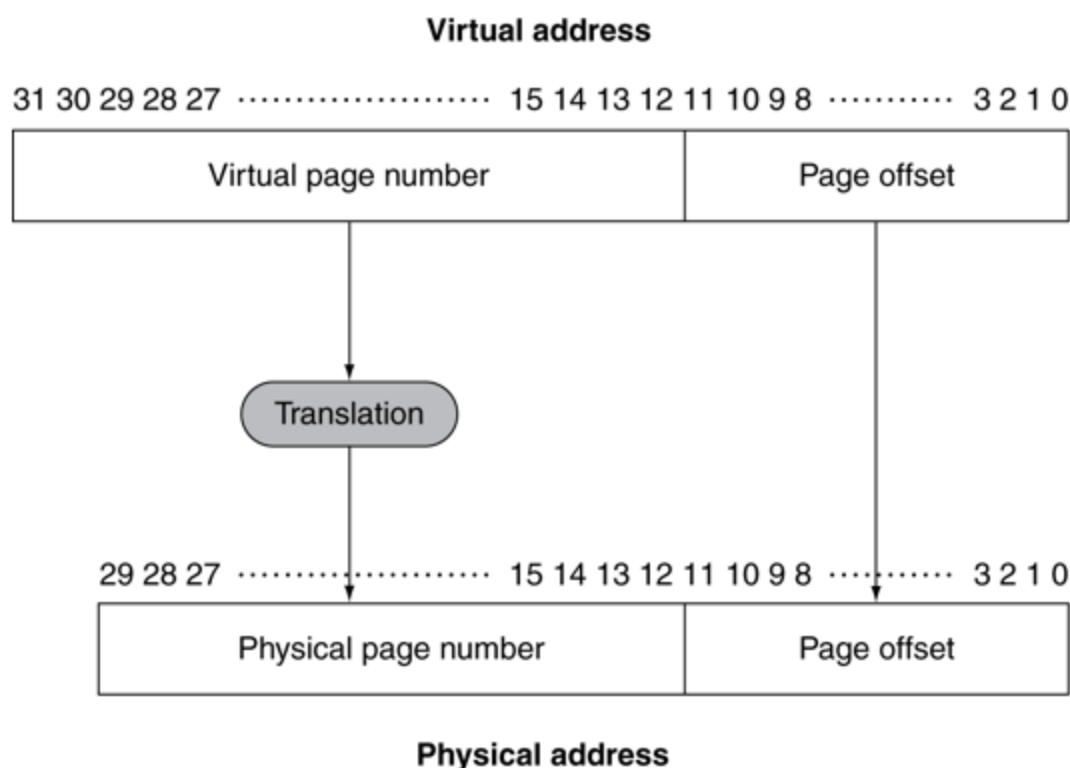
CPU时间 = 程序的指令数 × CPI × 时钟周期时间

$$= \frac{\text{程序的指令数} \times \text{CPI}}{\text{时钟频率}}$$

CPU时间 = CPU时钟周期数 × 时钟周期时间

$$= \frac{\text{CPU时钟周期数}}{\text{时钟频率}}$$

虚页号到物理页号的转换



物理地址的字地址、块地址、块内偏移

字地址（Word Address）：字地址表示数据在内存中的位置。每个字节有一个地址，而每个字通常由4个字节组成。因此，字地址可以通过物理地址除以4来计算。

$$\text{字地址} = \text{物理地址} / 4$$

块地址（Block Address）：块地址表示数据所在的缓存块的位置。块地址可以通过物理地址除以缓存块的大小来计算。

$$\text{块地址} = \text{物理地址} / \text{缓存块大小}$$

块内偏移（Block Offset）：块内偏移表示数据在缓存块内的位置。它可以通过物理地址对缓存块大小取模来计算。

块内偏移 = 物理地址 % 缓存块大小

cache 映射

直接映射

直接映射方式中，每个缓存块只能映射到缓存中的一个位置

块数 = 缓存总大小 / 缓存块大小

索引位数 = $\log_2(\text{块数})$

tag 位数 = 总位数 - index 位数 - 块内偏移位数

块内偏移位数 = $\log_2(\text{缓存块大小})$ （块大小为16字节，则需要4位表示块内偏移）

index | tag | offset

31 ——— 0

组相连

设有 n 路组相连

组数 = 缓存总容量 / (n * 缓存块大小)

索引位数 = $\log_2(\text{组数})$

块内偏移位数 = $\log_2(\text{缓存块大小})$

标记位数 = 总位数 - 索引位数 - 块内偏移位数

全相连

在全相联缓存中，每个缓存块可以映射到任意一个缓存位置，因此，每个缓存位置都可以存储不同的块。

块内偏移位数 = $\log_2(\text{缓存块大小})$

标记位数 = 总位数 - 块内偏移位数

由于缓存是全相联的，没有索引位，所以内存块可以映射到任意一个缓存块中

cache 缺失

理想 $CPI = 1$ ，I-cache 缺失率 a ，D-cache 缺失率 b ，缺失代价 c 个时钟周期，目标程序中访问内存的指令占 d

则

$$\hat{CPI} = CPI + ac + bdc$$

乘法器

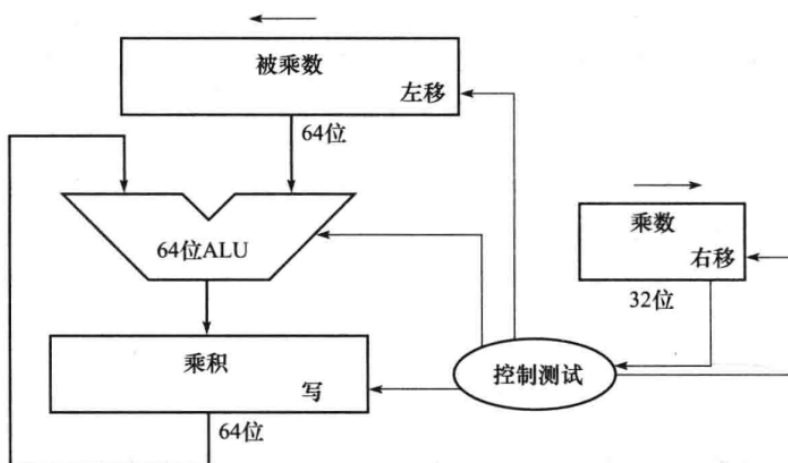


图 3-3 第一版乘法器硬件结构。被乘数寄存器、ALU 和积寄存器都是 64 位长，而乘数寄存器为 32 位长。（附录 B 对 ALU 进行了描述。）32 位的被乘数在开始时放置在被乘数寄存器的右半部分，然后每次左移一位。乘数则每次向相反的方向移动。算法开始时，积被初始化为 0。控制逻辑决定何时对被乘数和乘数寄存器进行移位，以及何时将新值写入积寄存器

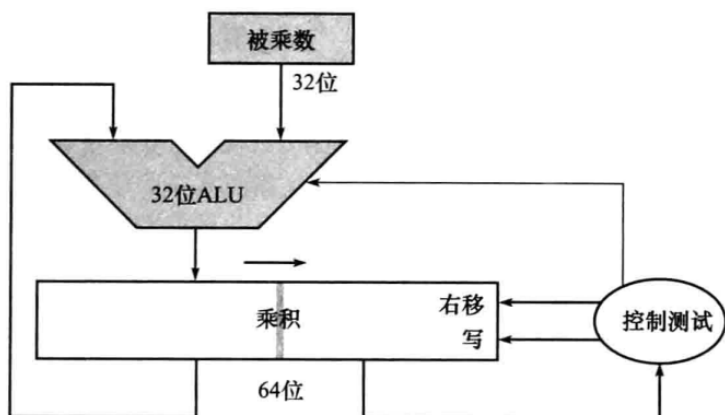


图 3-5 乘法器硬件的改进版。与图 3-3 中的第一版硬件结构相比，被乘数寄存器、ALU、乘数寄存器都是 32 位长，只有积寄存器是 64 位长。现在将积进行右移，单独的乘数寄存器也撤销了。乘数放在积寄存器的右半部分（乘法寄存器实际上应该是 65 位，以保存加法器的进位，但这里给出的是 64 位，以突出从图 3-3 的演变。）

除法器

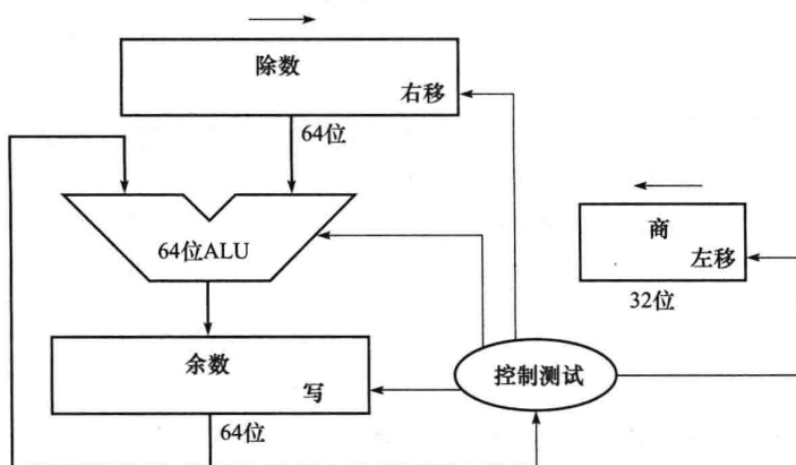


图 3-8 第一种除法器硬件结构。除数寄存器、ALU、余数寄存器都是 64 位宽，只有商寄存器是 32 位宽。32 位的除数开始放置在除数寄存器的左半部分，然后每次迭代右移一位。余数寄存器初始化为被除数。控制逻辑决定何时对除数和商寄存器进行移位以及何时将新值写入余数寄存器

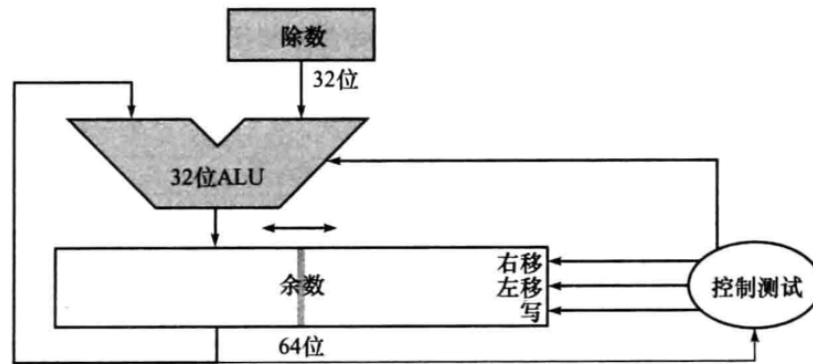


图 3-11 除法器的一种改进版本。除数寄存器、ALU、商寄存器都是 32 位，只有余数寄存器为 64 位。同图 3-8 相比，ALU 和除数寄存器都是位宽减半，余数进行左移。这个结构将商寄存器和余数寄存器的右半部分进行了拼接（正如图 3-5 中的那样，余数寄存器应该是 65 位从而保证加法器的进位不会丢失。） □

浮点加法器

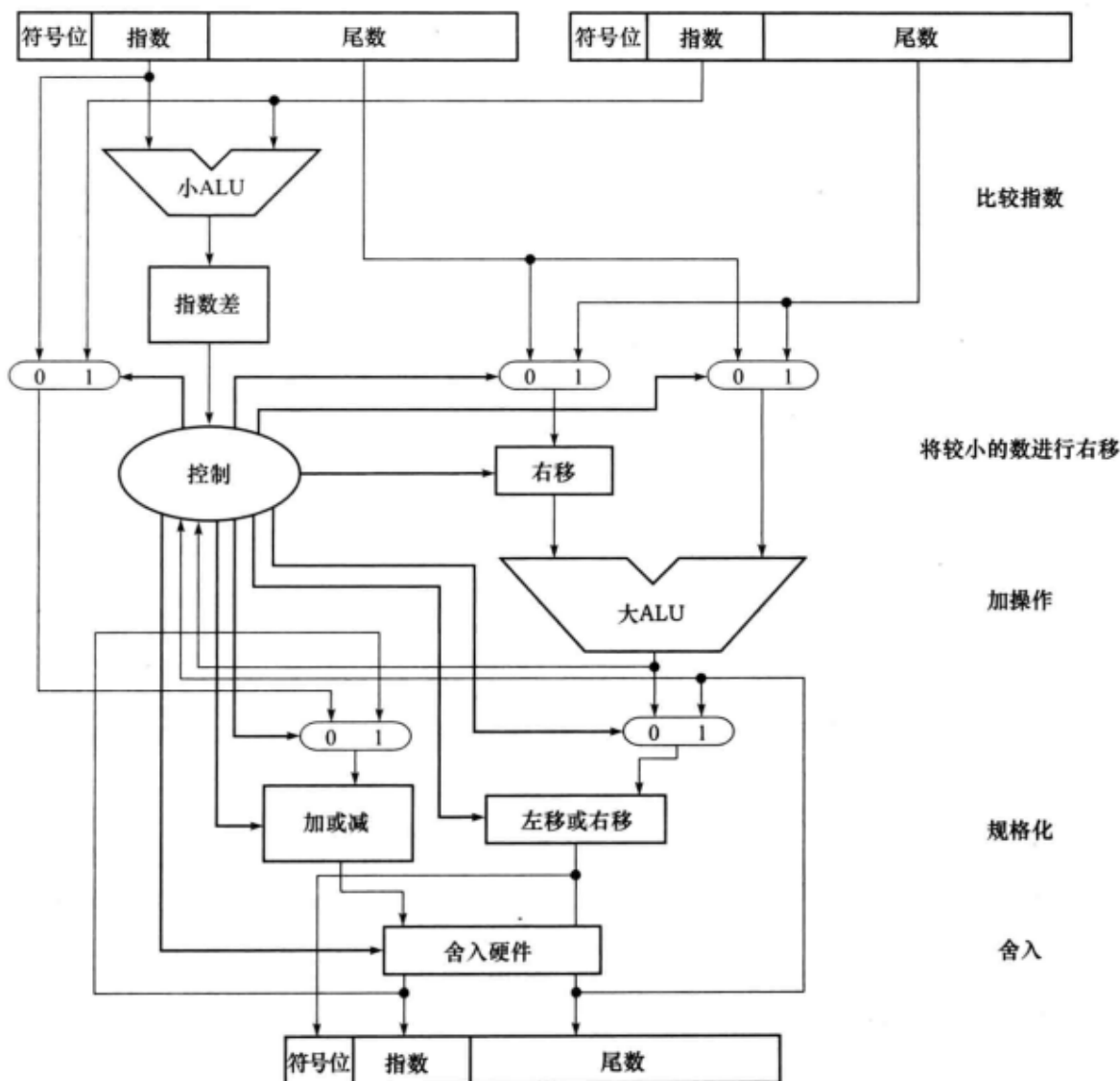


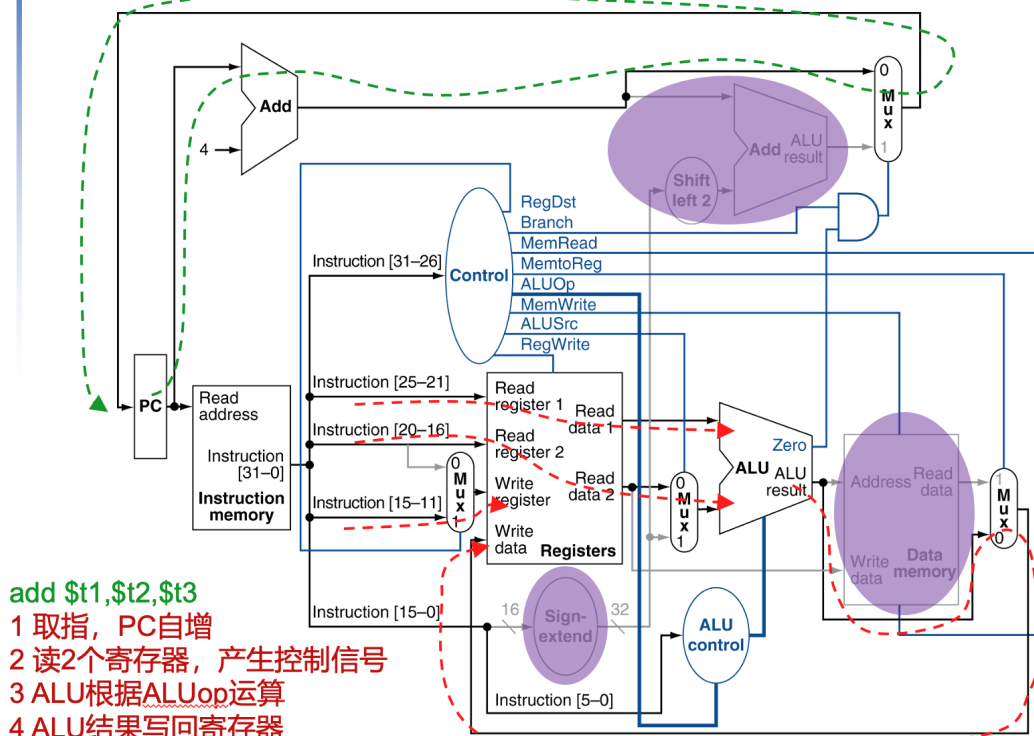
图 3-15 用于浮点加的算术单元的结构框图。图 3-14 的每步从顶向下对应到每个方框。首先，使用一个小的 ALU 将两个指数相减来决定哪个指数大及大多少。指数差将控制三个多路复用器；从左到右，选择出较大的指数、较小数的有效数和较大数的有效数。较小数的有效数通过右移后，和较大数的有效数用一个大的 ALU 相加。规格化步骤将和左移或者右移，同时增加或者减少指数。舍入产生最后的结果，这样也有可能需要再次规格化，然后产生最后的结果

单级流水线

Instruction	RegDst	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

Signal name	Effect when deasserted (0)	Effect when asserted (1)
RegDst	The register destination number for the Write register comes from the rt field (bits 20:16).	The register destination number for the Write register comes from the rd field (bits 15:11).
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, lower 16 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

R型指令



`add $t1,$t2,$t3`

1 取指, PC自增

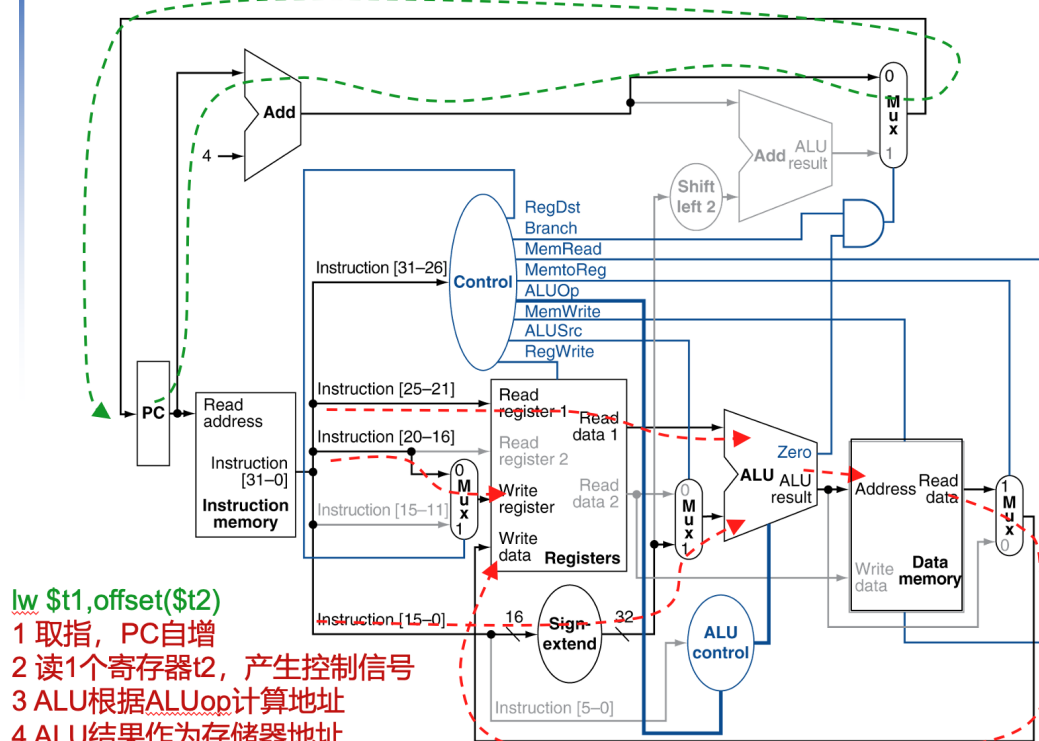
2 读2个寄存器, 产生控制信号

3 ALU根据ALUop运算

4 ALU结果写回寄存器

Chapter 4 — The Processor — 37

取字指令



lw \$t1,offset(\$t2)

1 取指, PC自增

2 读1个寄存器t2, 产生控制信号

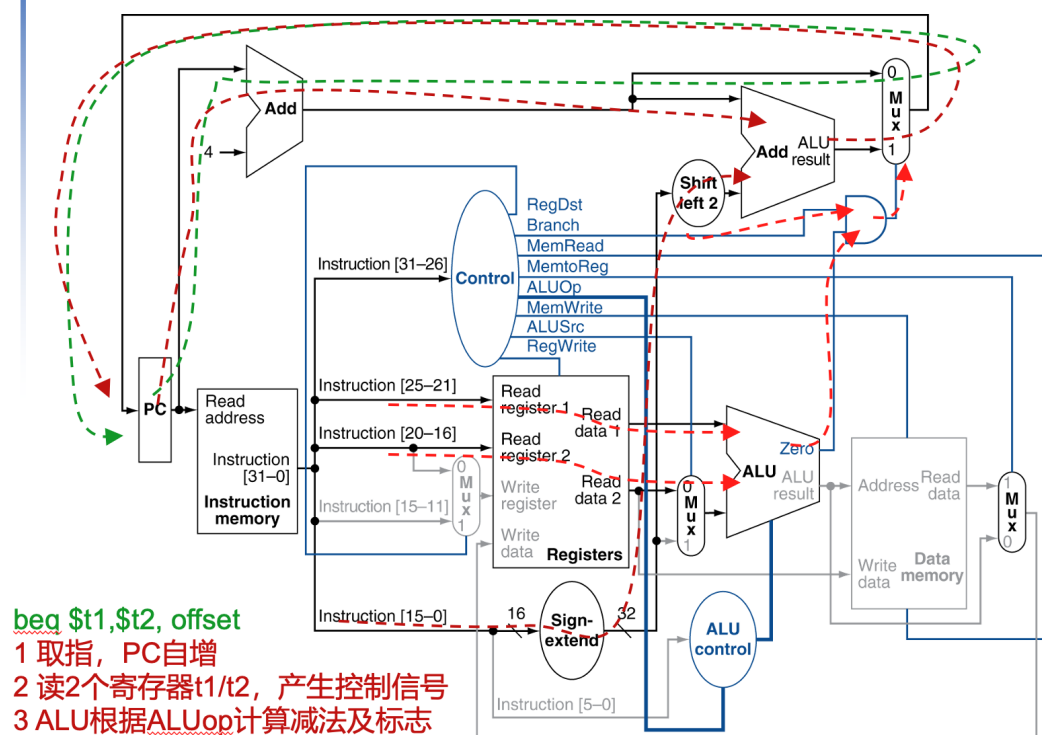
3 ALU根据ALUop计算地址

4 ALU结果作为存储器地址

5 数据存储器输出写入到t1

Chapter 4 — The Processor — 38

分支指令 beq



beq \$t1,\$t2, offset

1 取指, PC自增

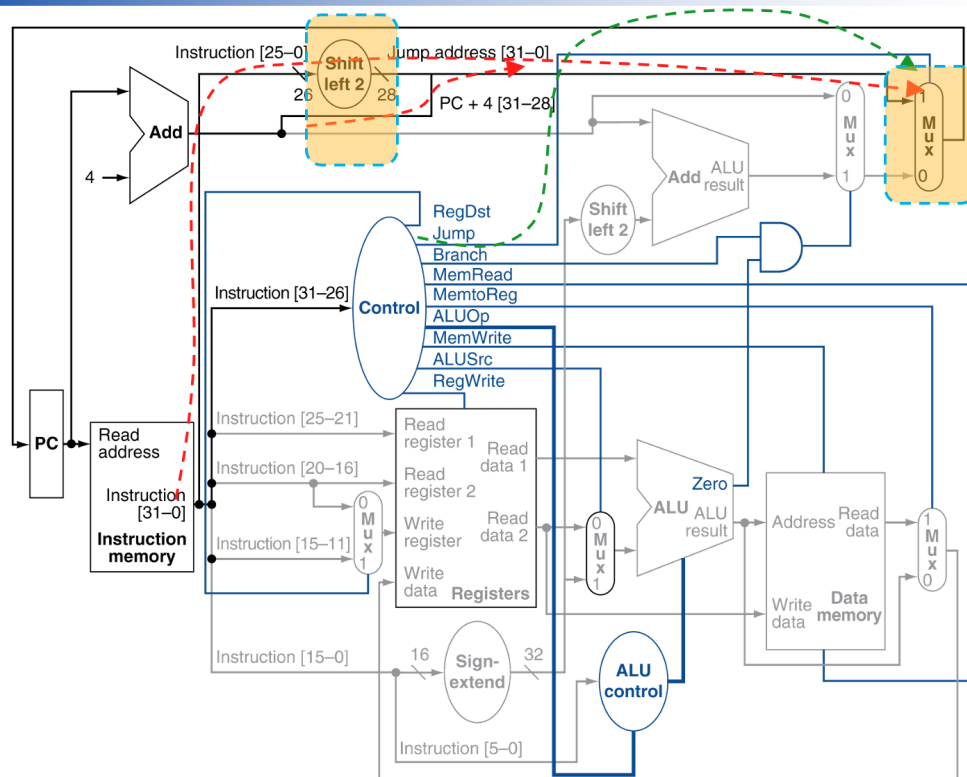
2 读2个寄存器t1/t2, 产生控制信号

3 ALU根据ALUop计算减法及标志

4 ALU结果标志位与branch产生PCsrc

Chapter 4 — The Processor — 39

增加跳转指令后的数据通路



Chapter 4 — The Processor — 42

五级流水线

Instruction	Execution/address calculation stage control lines				Memory access stage control lines			Write-back stage control lines	
	RegDst	ALUOp1	ALUOp0	ALUSrc	Branch	Mem-Read	Mem-Write	Reg-Write	Memto-Reg
R-format	1	1	0	0	0	0	0	1	0
lw	0	0	0	1	0	1	0	1	1
sw	X	0	0	1	0	0	1	0	X
beq	X	0	1	0	1	0	0	0	X

流水线控制信号

- Control signals derived from instruction
 - As in single-cycle implementation

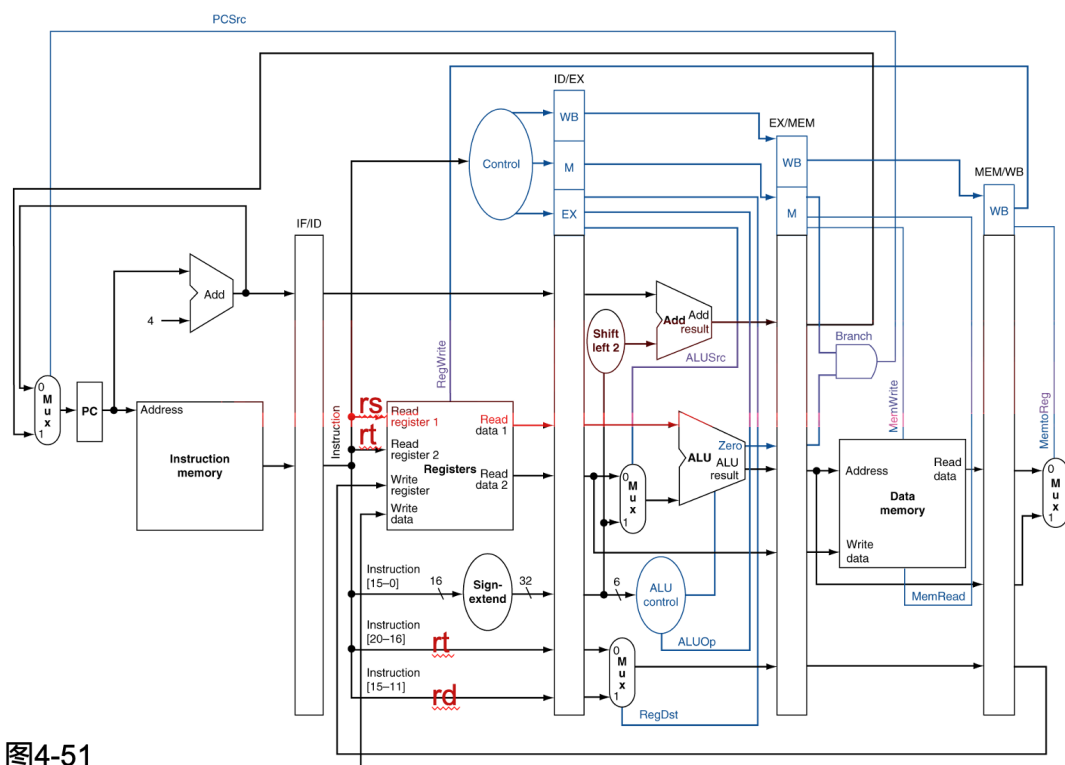
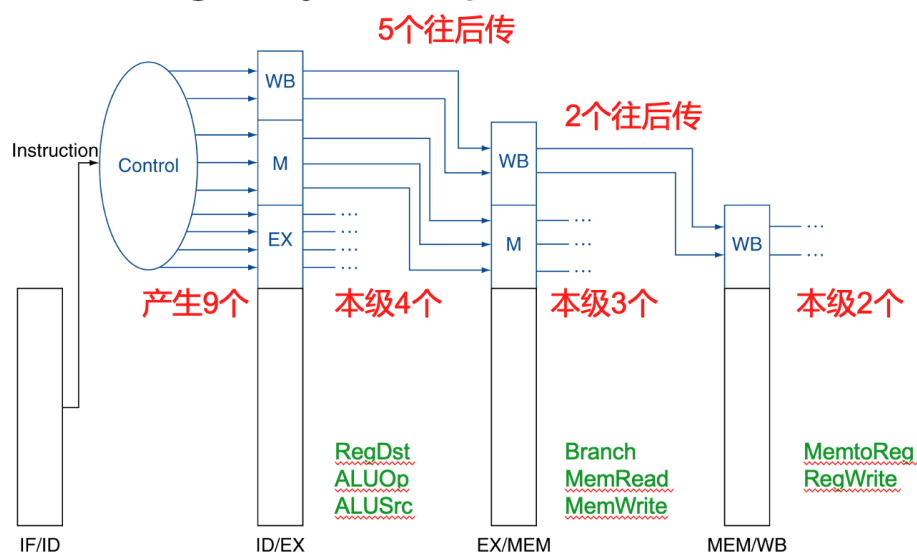


图4-51

带有前推功能的流水线

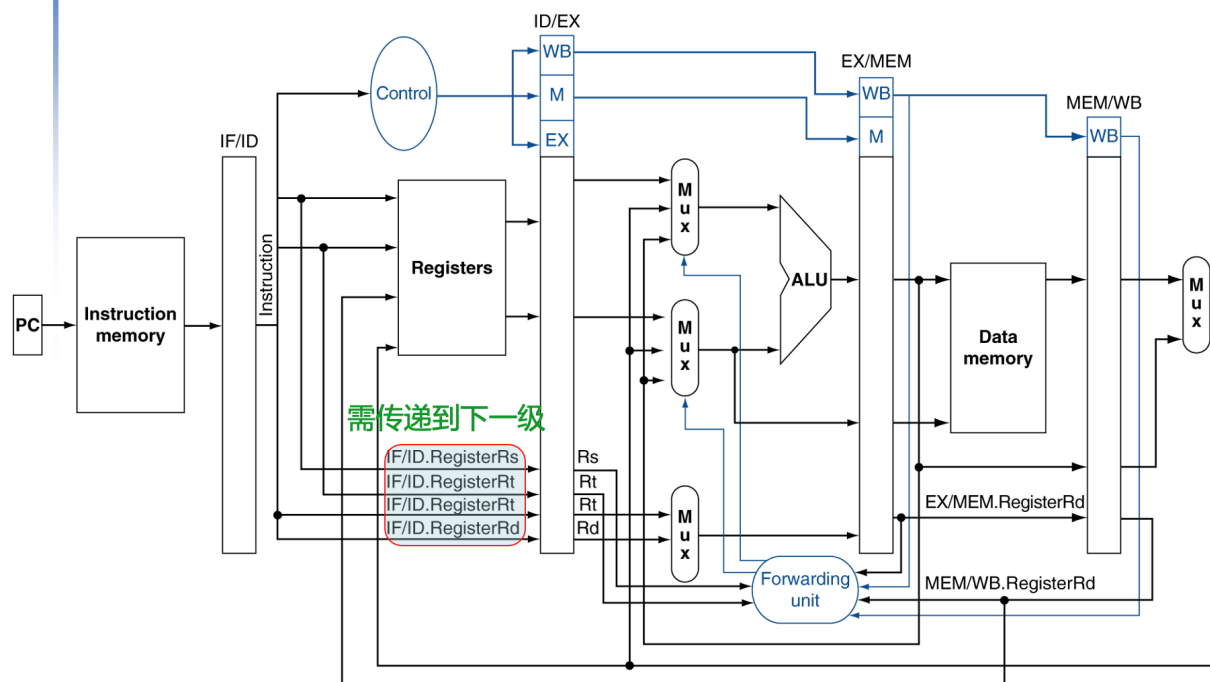


图4-56

带有冒险（阻塞）功能的流水线

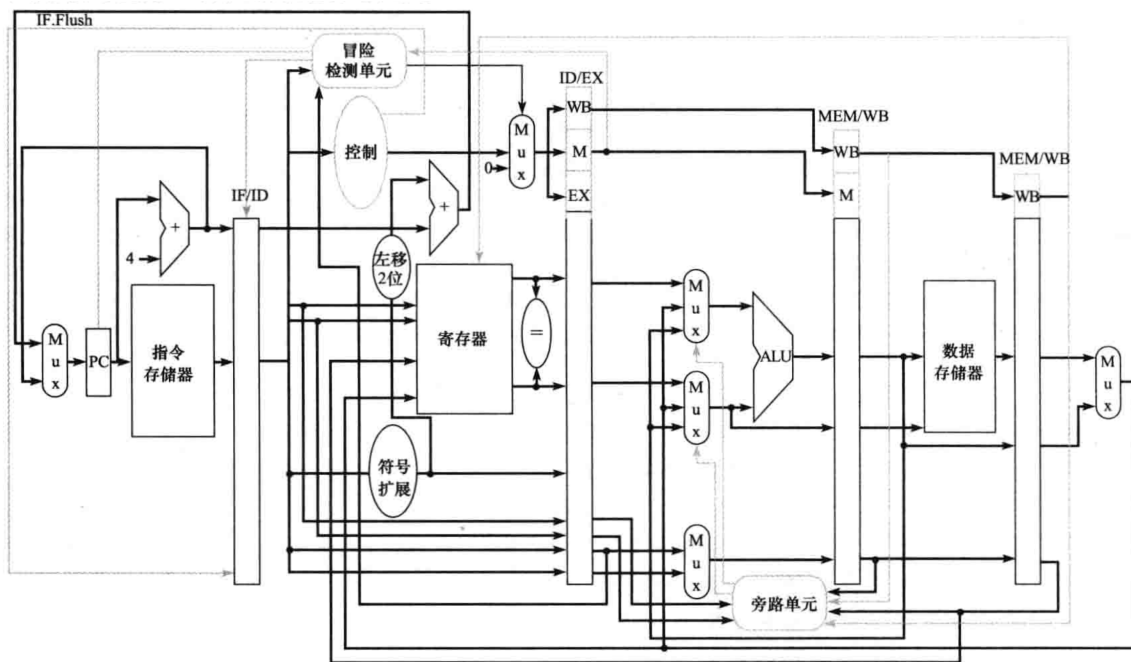
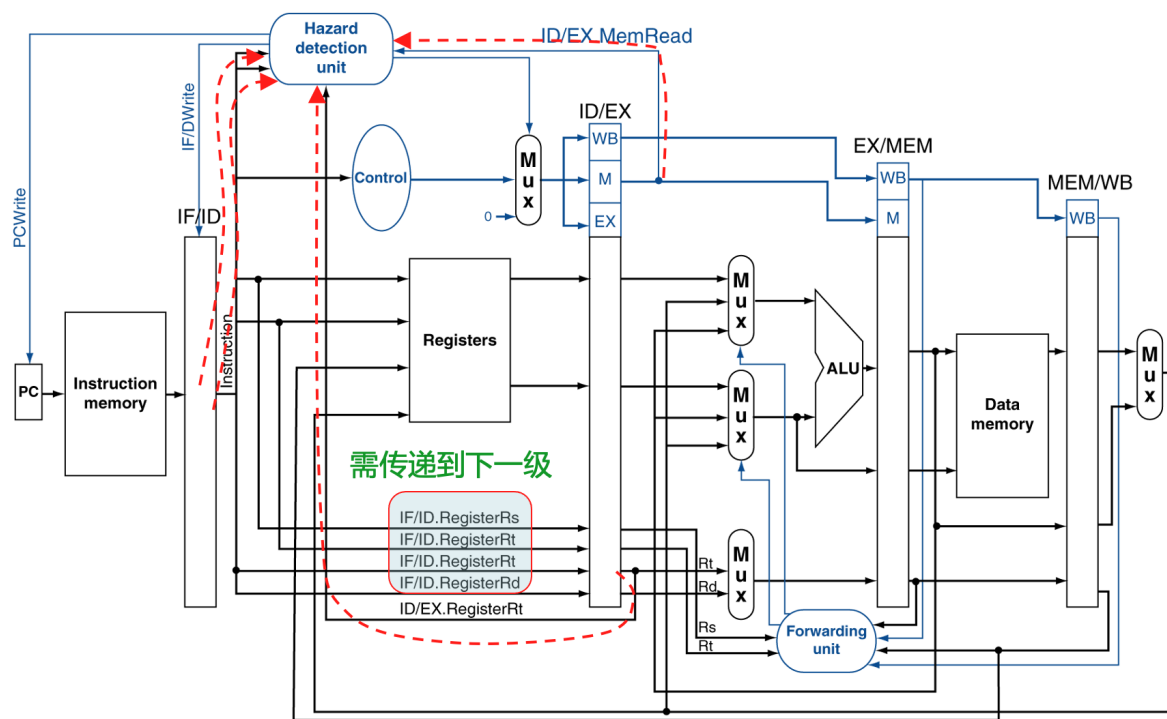


图 4-65 本章最终的数据通路与控制。注意，这是一个概略图，没有覆盖到数据通路的所有细节，如图 4-57 中的 ALUsrc 多选器和图 4-51 中的多选器控制都没有标识出来

数据冒险条件

1) EX 冒险:

```
if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd ≠ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs)) ForwardA = 10

if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd ≠ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRt)) ForwardB = 10
```

注意, EX/MEM. RegisterRd 字段是 ALU 指令 (来自 Rd 字段) 或装载指令 (来自 Rt 字段) 的目标寄存器号。

在这种情况下, 由于 MEM 级的结果是最新的, 因而结果是由 MEM 级旁路得到。这样, 对 MEM 冒险的控制策略为 (额外加入的条件采用粗体表示):

```
if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd ≠ 0)
and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
and (MEM/WB.RegisterRd = ID/EX.RegisterRs)) ForwardA = 01

if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd ≠ 0)
and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
and (MEM/WB.RegisterRd = ID/EX.RegisterRt)) ForwardB = 01
```

图 4-56 给出了为了支持旁路 EX 级结果所增加的必要硬件设备。注意, 图中 EX/MEM. RegisterRd 字段是一条 ALU 指令 (来自 Rd 字段) 或装载指令 (来自 Rt 字段) 的目标寄存器。

- ID阶段给出的ALU输入寄存器为:
 - IF/ID.RegisterRs, IF/ID.RegisterRt
- “取数-使用” 冒险判定
 - ID/EX.MemRead and
((ID/EX.RegisterRt = IF/ID.RegisterRs) or
(ID/EX.RegisterRt = IF/ID.RegisterRt))
- 发现该类型冒险后, 需要阻塞插入气泡
 - 然后等待旁路电路处理, 否则还需一个阻塞

分支延迟槽

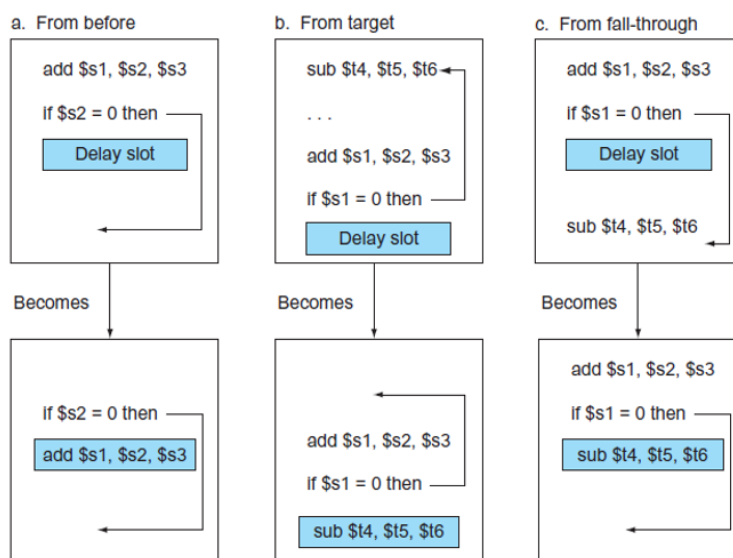
延迟槽技术

- 分支发生在分支指令之后的第二个时钟周期

- 中间间隔一个“分支延迟槽”可以执行无关指令

- 无关指令：

- 前面的指令
- 后续指令
- 目标处指令



Chapter 4 — The Processor — 126

跳转寻址方式

下一条指令： $PC = PC + 4$

`beq`：目标地址 = $PC + \text{offset} * 4$ ，跳转范围 $PC + [-2^{17}, 2^{17} - 4]$

`j, jal`：目标地址 = $PC[31 : 28] : (\text{address} * 4)$ ，跳转范围 2^{28}

`jr`：目标地址 = $\$ra$ ，跳转范围 2^{32}

异常

把PC存到EPC，记录原因Cause，跳转到处理程序

处理操作

- 读出异常的原因, 跳转到相应的处理代码
- 确定所需的处理
- 如果问题可以解决
 - 解决问题
 - 返回到EPC指出的被中断指令
- 否则
 - 终止程序
 - 报告EPC, cause, ...

数据冒险阻塞

前推的情况：

lw：前推+阻塞1个周期

add：直接前推

不前推的情况：

lw：阻塞2个周期（读wb）

add：阻塞2个周期（读wb）